University of Dhaka
Department of Robotics and Mechatronics Engineering

Reinforcement Learning for Mars Rover Wheel Entrapment Recovery in Granular Regolith

A Research Project Report

Authors: Meraj Hossain Promit
Chandak Chakma

Program: B.Sc. in Robotics and Mechatronics Engineering

Supervisor: *[Supervisor Name]*

April 2026

Abstract

This report presents a reinforcement learning (RL) framework for autonomous recovery of Mars rovers from wheel entrapment in granular regolith. We leverage the Newton physics engine with Material Point Method (MPM) [21] for high-fidelity granular soil simulation, Isaac Lab [9] for articulated robot simulation, and Proximal Policy Optimization (PPO) [2] with recurrent GRU networks [13] for policy training via the skrl library [10]. The system addresses the critical challenge of wheel entrapment faced by planetary exploration rovers, as exemplified by NASA's Spirit rover incident in 2009 [14], by training escape policies in a physically realistic simulation environment.

Our approach features a 29-dimensional proprioceptive observation space incorporating wheel velocities, slip ratios, IMU data, drive torques, and dual-sensor entrapment detection flags. The policy is trained across 64 parallel environments with curriculum learning that progressively deepens wheel sinkage from 8 cm to 15 cm. Preliminary results over 200,000 timesteps on an RTX 5070 Ti demonstrate an 80% escape rate, mean forward velocity of 0.25 m/s, and full curriculum completion. The trained policy exhibits emergent escape behaviors including rocking maneuvers and coordinated wheel–steering actions.

Future work includes scaling training to 2M+ timesteps, developing a CNN-GRU sinkage detection classifier, and deploying the trained policy on a Raspberry Pi 5 via ONNX Runtime for real-time inference on physical hardware.

Contents

List of Figures

List of Tables

1 Introduction

On May 1, 2009, NASA's Spirit rover drove into soft sulfate-rich soil and never moved again. Eight months of recovery attempts, thousands of ground-based tests, and 150 million kilometres of communication delay could not save it. This project asks: **what if the rover could have saved itself?**

1.1 Problem Statement

Planetary exploration rovers operating on Mars and other celestial bodies [15, 16, 17] face a critical operational challenge: **wheel entrapment in granular regolith**. When a rover's wheels sink into loose sand or soil, the vehicle loses mobility and may become permanently immobilized. This problem is particularly acute for several reasons:

- **Rocker-bogie suspension**: While excellent for rough terrain, it distributes weight in ways that can cause individual wheels to sink disproportionately in soft soil.
- **Sandy and cratered terrain**: The Martian surface contains extensive dune fields, impact craters filled with fine regolith, and wind-blown sand deposits that present persistent mobility hazards.
- **Long communication delays**: With Earth–Mars one-way latencies of 4–24 minutes, real-time teleoperation for recovery is impractical, necessitating fully autonomous solutions.
- **Extended mission lifetimes**: As missions extend beyond planned durations, rovers inevitably encounter terrain types not anticipated during original mission planning.

Wheel entrapment is formally characterised by the simultaneous satisfaction of three conditions:

$$f_{\mathrm{ent}} = 1 \iff \begin{cases} |v_x| < 0.15 \text{ m/s} & \text{(near-zero forward velocity)} \\ \bar{\sigma}_{\mathrm{slip}} > 0.4 & \text{(high mean wheel slip)} \\ d_{\mathrm{sinkage}} > 0 & \text{(positive wheel sinkage)} \end{cases} \tag{1}$$

where v_x is the body-frame forward velocity, $\bar{\sigma}_{\mathrm{slip}} = \frac{1}{6}\sum_i \sigma_i$ is the mean slip ratio across all six drive wheels, and d_{sinkage} is the wheel sinkage depth into the granular medium. All three conditions must hold for at least **15 consecutive steps** (0.6 s) before the entrapment flag f_{ent} is raised.

1.2 Problem Definition

The entrapment recovery problem is formulated as a **partially observable Markov decision process (POMDP)**. At each timestep t, the rover receives a proprioceptive observation $\mathbf{o}_t \in \mathbb{R}^{29}$ and must select an action $\mathbf{a}_t \in \mathbb{R}^{10}$ to maximise the expected discounted return:

$$J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \gamma^t R(\mathbf{s}_t, \mathbf{a}_t)\right], \qquad \gamma = 0.99 \tag{2}$$

The shaped reward R is the sum of seven terms — three incentives and four penalties — designed to reward escape while discouraging dangerous manoeuvres:

$$R_t = \underbrace{r_{\text{progress}} + r_{\text{escape}} + r_{\text{rocking}}}_{\text{incentives}} - \underbrace{p_{\text{slip}} + p_{\text{tilt}} + p_{\text{smooth}} + p_{\text{abnormal}}}_{\text{penalties}} \tag{3}$$

Term	Weight	Formula & Description
Incentives		
r_{progress}	+1.5	$v_x \cdot \Delta t \cdot (1 - f_{\text{ent}})$ — forward velocity reward, suppressed when trapped so rocking takes over
r_{escape}	+20.0	Sparse milestones at 0.5 m (×0.2), 1.0 m (×0.4), 1.5 m (×1.0) scaled by τ_{time}; plus dense shaping $0.5\, d\, \Delta t$ per step
r_{rocking}	+2.0	$\Delta v_x \cdot f_{\text{ent}} \cdot \Delta t$ — rewards velocity change during entrapment to encourage active rocking manoeuvres
Penalties		
p_{slip}	0.5	$\bar{\sigma}_{\text{slip}} \cdot (1 - f_{\text{ent}}) \cdot \Delta t$ — penalises excessive wheel slip during normal driving
p_{tilt}	0.3	$\lVert \boldsymbol{\omega}_{xy} \rVert_2 \cdot \Delta t$ — penalises roll/pitch angular velocity to maintain rover stability
p_{smooth}	0.05	$\lVert \mathbf{a}_t - \mathbf{a}_{t-1} \rVert_2 \cdot \Delta t$ — penalises jerky action changes to encourage smooth control
p_{abnormal}	0.3	$(-v_x)^+ \cdot f_{\text{anomaly}} \cdot (1 - f_{\text{ent}}) \cdot \Delta t$ — penalises backward motion under torque anomaly when not trapped

f_{ent}: entrapment flag; f_{anomaly}: torque anomaly flag; d: distance from spawn; $\Delta t = 0.04\,\text{s}$

Table 1: Reward function v4 used during PPO training. The escape bonus (weight 20) dominates during entrapment, while the rocking bonus encourages active self-extraction. Compared to Bi & Ding (2026) [1], who use GAIL-fused [31] rewards ($R = 0.9R_H + 0.1R_D$) with expert demonstrations, our reward is fully handcrafted and designed for 6-DOF steering under Martian gravity without requiring any expert data.

The partial observability arises because the rover cannot directly sense the depth of sinkage per wheel, the local terrain composition, or its position relative to the entrapment zone boundary. This motivates a **recurrent policy** based on the Gated Recurrent Unit (GRU) [13] that maintains a hidden state $\mathbf{h}_t$ to implicitly estimate these unobserved variables from the history of observations. GRUs are a lightweight alternative to LSTMs [32] that achieve comparable performance on temporal sequence modeling with fewer parameters.

1.3 Motivation: The Spirit Rover Incident (2009)

The most prominent example of wheel entrapment in planetary exploration occurred on May 1, 2009, when NASA's Mars Exploration Rover *Spirit* became embedded in soft sulfate-rich regolith at a site named "Troy" near the western edge of Home Plate in Gusev Crater [14, 15]. Figure 1 illustrates the timeline of this critical incident.

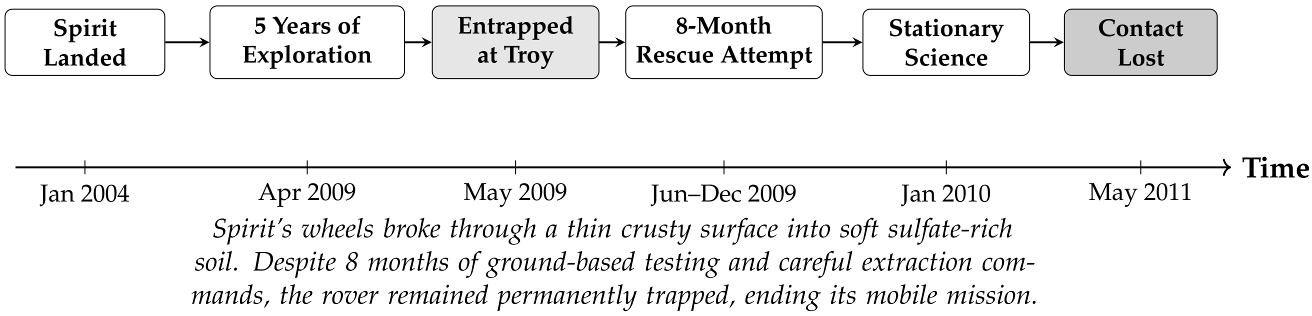


Figure 1: Timeline of the Spirit Rover entrapment incident. After 5 years of successful exploration, Spirit became permanently immobilized in soft regolith, motivating autonomous entrapment recovery research.

Key lessons from the Spirit incident that inform this research:

1. **Detection was delayed**: Spirit had driven partway into the soft soil before operators recognized the hazard, as slip ratio was not monitored in real-time.

2. **Recovery required months**: The 8-month extraction campaign involved extensive Earth-based testing on rover replicas, constrained by communication delays.

3. **Autonomous response was absent**: Spirit had no onboard capability to detect or respond to entrapment autonomously.

4. **Granular physics are critical**: The behavior of the sulfate-rich soil was poorly understood, leading to extraction commands that inadvertently worsened the entrapment.

1.4 Objectives

This research aims to develop a complete autonomous entrapment detection and recovery system for Mars rovers. The five specific objectives are:

1. Realistic Simulation — Build a high-fidelity simulation environment coupling granular material physics (Newton MPM) with articulated rover dynamics (Isaac Lab) under Martian gravity ($3.72\,m/s^2$), enabling realistic wheel–soil interaction for RL training.

2. Entrapment Detection — Develop a dual-sensor detection scheme combining slip-ratio monitoring and drive-torque anomaly detection for robust, real-time entrapment classification without external sensing or GPS.

3. Recovery Policy — Train a recurrent RL policy (PPO + GRU) that autonomously discovers and executes effective escape manoeuvres — including rocking, steering pivots, and differential drive — from varying degrees of wheel sinkage (6–12 cm curriculum).

4. Curriculum Learning — Implement progressive difficulty scheduling that begins with shallow sinkage and gradually increases entrapment severity, enabling the policy to learn robust escape strategies that generalise across terrain conditions.

5. Sim-to-Real Transfer — Establish a complete deployment pipeline — domain randomisation during training, ONNX export of the trained GRU policy, and real-time inference on a Raspberry Pi 5 — bridging the simulation–reality gap for onboard autonomous operation.

2 Related Work

This section reviews prior work in three key areas: (1) wheeled robot entrapment and escape strategies, (2) granular media simulation for robotics, and (3) reinforcement learning for locomotion and recovery.

2.1 Escape Strategies for Wheeled Robots

2.1.1 Bi and Ding (2026): DRL-Based Escape for Skid-Steered Mobile Robots

Bi and Ding [1] presented a deep reinforcement learning framework for escape and path point tracking of skid-steered mobile robots (SSMRs) under undulating terrain conditions in outdoor orchards. Their work is the most directly relevant to ours.

Description. The authors developed a two-phase system: (1) an abnormal state detection module using motor torque analysis, and (2) a DRL-based escape policy using PPO with LSTM networks [32]. Their key innovation is a *reward fusion* strategy combining handcrafted rewards with GAIL-derived [31] (Generative Adversarial Imitation Learning) discriminator rewards: $R = 0.9R_H + 0.1R_D$, where R_H includes path tracking, abnormal action, and conditional rewards, and R_D comes from a discriminator trained on expert demonstrations. The system was trained for 4 million timesteps in a Unity simulation environment with extensive domain randomization (20+ parameters, see their Table 9) and validated in real-world outdoor orchard conditions, achieving 90% escape success rate in both simulation and field tests.

Their observation space comprised 30 dimensions organized as $\mathbf{s} = \{\mathbf{s}_{\text{motor}}, \mathbf{s}_{\text{motion}}, \mathbf{s}_{\text{position}}, s_{\text{abnormal}}\}$: 16D motor data (set speed, current speed, current torque, average torque $\times$ 4 motors), 7D motion state (quaternion attitude + angular velocity), 2D position (distance and angle to target), and 1D anomaly flag. The action space is 2D: left and right wheel velocities $\mathbf{a} = [\omega_l, \omega_r]$.

Strengths.

- Achieved 90% escape success rate in both simulation and real-world tests, demonstrating strong sim-to-real transfer via domain randomization.
- LSTM-based temporal modeling captures the sequential nature of entrapment dynamics, enabling memory-dependent escape strategies.
- GAIL reward fusion ($R = 0.9R_H + 0.1R_D$) combines human intuition with learned reward shaping, producing smoother and more natural escape trajectories compared to pure handcrafted or pure imitation approaches.
- Comprehensive anomaly detection based on motor torque threshold (Eq. 10 in their paper): $\frac{1}{c_t}\sum_{n=t-c_t}^{t}\frac{|^{i}T_n|}{T_N} > T_{\text{thread}}$ with $T_{\text{thread}} = 0.95$.
- Validated on real outdoor terrain (orchards) with undulating surfaces using UWB positioning.

Weaknesses.

- Limited to 4-wheel skid-steered platforms; does not address the more complex kinematics of 6-wheel rocker-bogie suspension systems used in Mars rovers (Curiosity, Perseverance).
- Uses rigid terrain simulation (Unity) and does not model granular soil mechanics (particle flow, sinkage dynamics, bulldozing resistance) that are critical for planetary regolith interaction.
- Tested under Earth gravity ($9.81\,\text{m/s}^2$), not Mars gravity ($3.72\,\text{m/s}^2$), which significantly affects soil–wheel traction and sinkage behavior.
- Requires expert demonstrations for GAIL training, which may not be available for novel Mars terrain conditions.
- Two-dimensional action space (ω_l, ω_r) does not leverage independent steering joints for more sophisticated escape maneuvers.
- Entrapment is caused by tire lift-off on undulating terrain, not by granular sinkage, which is a fundamentally different failure mode.

2.1.2 Inotsume et al. (2020): Slip Prediction for Planetary Rovers

Description. Inotsume et al. [3] investigated real-time slip prediction for planetary rovers using proprioceptive sensing and terrain parameter estimation. A Kalman filter fuses motor current, wheel sinkage estimates, and terrain slope to predict slip ratio based on Bekker–Wong terramechanics theory.

Strengths.

- Principled physics-based approach grounded in terramechanics theory; provides interpretable terrain parameter estimates.
- Runs in real-time on embedded hardware with low computational requirements.
- Directly applicable to Mars rover missions with existing sensor suites.

Weaknesses.

- Focuses on slip prediction only; does not address active recovery from entrapment.
- Requires careful calibration for each terrain type, limiting generalization to unknown Martian soils.
- Simplified Bekker–Wong model may not capture complex particle–wheel coupling during deep sinkage.

2.1.3 Ding et al. (2011): Wheel–Terrain Interaction Modeling

Description. Ding et al. [4] presented a comprehensive wheel–soil interaction model for planetary rovers, deriving analytical expressions for drawbar pull, rolling resistance, and sinkage as functions of wheel geometry, soil parameters (cohesion, internal friction angle, shear deformation modulus), and slip ratio. The model accounts for multi-pass effects where trailing wheels encounter pre-compacted soil.

Strengths.

- Rigorous analytical framework validated experimentally with single-wheel testbed data.
- Accounts for multi-wheel interactions and soil compaction by preceding wheels.

Weaknesses.

- Steady-state analysis only; does not capture transient dynamics during entrapment onset or recovery.
- Assumes homogeneous soil properties, unrealistic for heterogeneous Martian terrain.
- Does not address control strategies for escape from entrapment.

2.1.4 Jain et al. (2019): RL for Planetary Rover Locomotion

Description. Jain et al. [5] applied DQN for adaptive locomotion of planetary rovers, learning to select discrete gaits (forward, turn, reverse) based on terrain features from onboard cameras in Gazebo simulation.

Strengths. Demonstrated that RL can learn terrain-adaptive locomotion; uses visual observations relevant to missions.

Weaknesses. Discrete action space limits expressiveness of recovery maneuvers; does not specifically address entrapment; approximate terrain deformation model.

2.1.5 Fankhauser et al. (2018): ANYmal Recovery from Falls

Description. Fankhauser et al. [6] developed a recovery controller for the ANYmal quadruped robot combining optimization-based motion planning with learned recovery policies. The multi-phase architecture (detect $\rightarrow$ recover $\rightarrow$ resume) provides a useful design template.

Strengths. Successful sim-to-real transfer for dynamic recovery behaviors; multi-phase architecture is applicable to our problem.

Weaknesses. Designed for legged robots with body reconfiguration; wheel entrapment in granular media presents fundamentally different physical constraints.

2.1.6 Mortensen and Bøgh (2024): RLRoverLab RL Suite for Planetary Rovers

Mortensen and Bøgh [7] introduced RLRoverLab, an open-source reinforcement learning suite for simulating and training planetary rovers on extraterrestrial bodies. This work is directly relevant because our project builds upon the AAU Mars Rover assets, training frameworks, and pre-trained navigation policies provided by RLRoverLab.

Description. RLRoverLab is implemented on top of the Nvidia ORBIT framework [39], which uses Isaac Sim as the physics and graphics engine. The suite provides: (1) four terrain types (flat, hill, obstacle, maze) with procedural generation; (2) three rover models: the AAU Rover Complex (6-wheel, rocker-bogie, ~1 m footprint), AAU Rover Simple (same dimensions, no suspension), and ExoMy (open-source 3D-printable 6-wheel rover) [40]; (3) four steering modes (Ackermann, skid, crab, direct joint mapping); (4) two tasks (autonomous navigation, grasping); and (5) a data recording system for offline RL and imitation learning. Benchmarks in the paper compare PPO, TRPO [26], and TD3 on the navigation task, showing PPO converges fastest.

Connection to our work. We use the AAU Rover Complex model (USD assets from RL-RoverLab) as our physical platform. In the envisioned full system (*dual-brain policy*, Section 5), the RLRoverLab navigation policy acts as the *Navigation Brain* during normal traversal; our GRU-PPO escape policy takes over as the *Escape Brain* upon entrapment detection and returns control once the rover is free. This clean separation of concerns makes RLRoverLab an ideal complement to our contribution.

Strengths.

- Provides high-quality AAU Rover USD assets with full rocker-bogie kinematics, directly reused in our entrapment environment.
- Modular design (swap terrain, robot, steering mode, task) reduces engineering effort for downstream research.
- Pre-trained navigation policies serve as the *normal-traversal baseline* in our dual-brain architecture.
- Built on ORBIT/Isaac Sim: seamless integration with our Isaac Lab [9] training stack.

Weaknesses.

- No granular terrain modeling: all terrains use rigid-body collision meshes. Wheel–soil interaction is not physically modeled, making it unsuitable for studying entrapment.
- Navigation task only: no entrapment detection, recovery policy, or terramechanics-aware reward shaping.
- Flat and procedural terrains do not replicate fine-grained Martian regolith properties (cohesion, friction angle, sinkage modulus).

2.2 Granular Media Simulation

The Material Point Method (MPM) [21] has emerged as the gold standard for simulating granular materials in robotics. Originally developed for snow [22] and extended to sand with Drucker-Prager elastoplasticity [23], MPM treats granular media as a continuum discretized into particles that are transferred to a background grid for force computation. Unlike position-based dynamics (PhysX PBD) [25] which overestimates traction, MPM correctly models Coulomb friction, yield surface behavior, sinkage dynamics, and bulldozing resistance [24]. The classic Bekker–Wong terramechanics theory [18] provides analytical predictions of wheel sinkage and drawbar pull, but is limited to steady-state conditions and homogeneous soils; MPM captures the transient dynamics critical for entrapment onset and recovery. The Newton physics engine [8] provides GPU-accelerated MPM through `SolverImplicitMPM`, enabling thousands of particles per environment with real-time training performance.

2.3 Summary and Comparison

Table 2 positions our contribution relative to the reviewed literature.

Table 2: Comparison of related works on wheeled robot entrapment and recovery

Work	Robot	Key Limitation	Terrain	Method	Gravity	Escape
Bi & Ding (2026)	4-wheel SSMR	No granular physics; Earth only; needs expert demonstrations for GAIL	Rigid (Unity)	PPO+LSTM +GAIL	Earth	90%
Inotsume (2020)	6-wheel rover	Detection only; no recovery policy trained	Bekker model	Kalman filter	Mars	N/A
Ding (2011)	Multi-wheel	Steady-state model only; no real-time closed-loop control	Analytical	Physics model	Any	N/A
Jain (2019)	6-wheel rover	Discrete action space; no granular physics coupling	Approx.	DQN	Mars	N/A
Fankhauser (2018)	ANYmal (legged)	Legged robot; constraints not applicable to wheeled entrapment	Rigid	Opt.+RL	Earth	N/A
Ours	**6-wheel rocker-bogie**	**Early training (200K steps); sim-to-real transfer not yet validated in hardware**	**MPM granular**	**PPO+GRU**	**Mars**	**80%**

3 Methodology

This section details the complete methodology, divided into work already completed and expected future work.

Figure 2 presents the overall escape policy generation method for our Mars rover system.

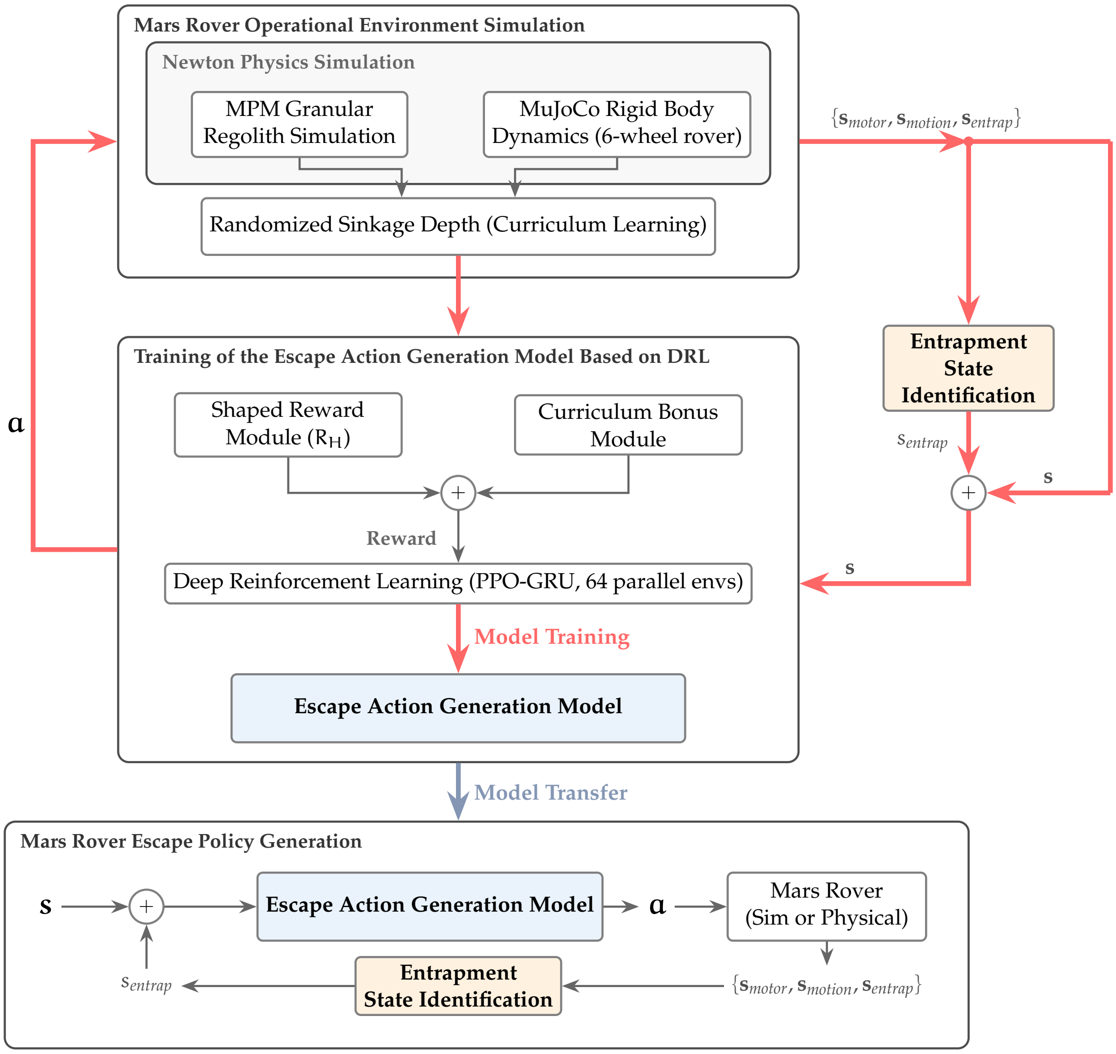


Figure 2: Escape policy generation method. Top: the training pipeline in simulation with reward shaping and curriculum learning. Bottom: the deployment pipeline on the physical or simulated Mars rover.

Figure 3 shows the simulation framework for our 6-wheel Mars rover with MPM granular terrain.

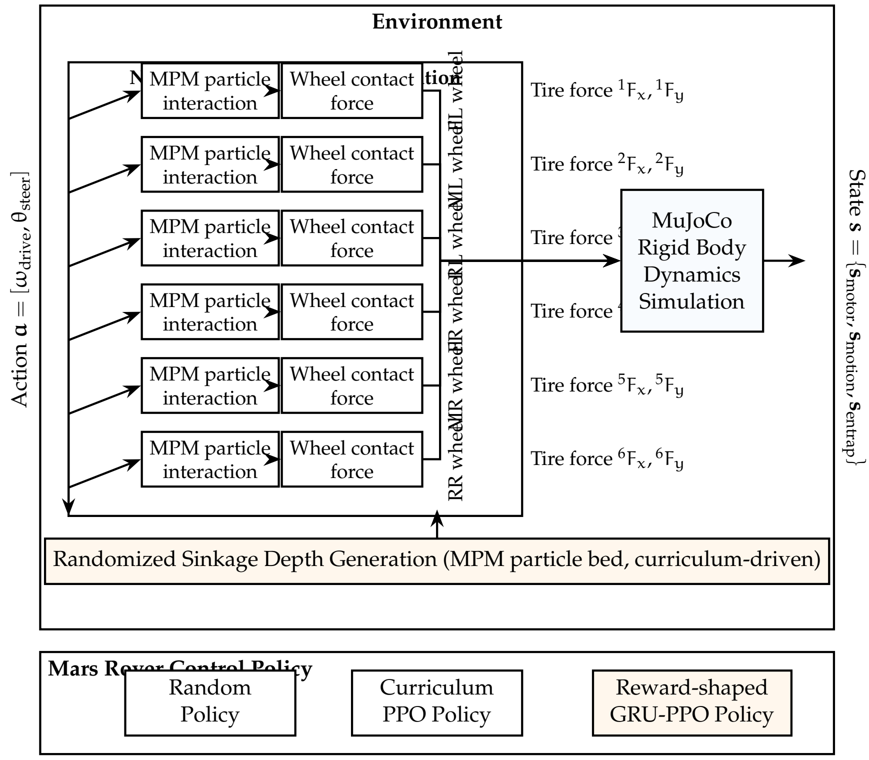


Figure 3: Simulation framework. The framework couples 6-wheel MPM particle interactions with granular sinkage via Newton MPM, and incorporates curriculum-driven sinkage depth randomization.

Figure 4 shows the control policy network structure. We use GRU for reduced complexity, with 29D input and 10D actions (6 drive + 4 steer).

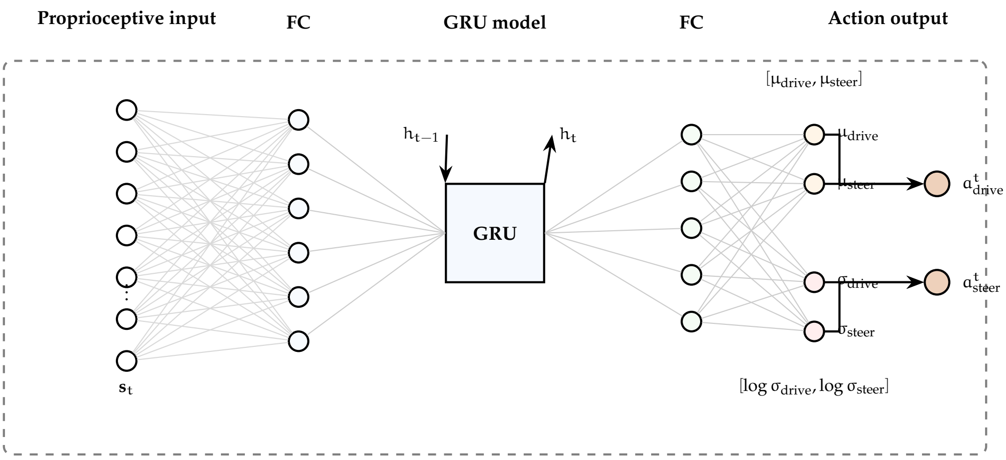


Figure 4: The structure of the Mars rover control policy network. GRU (256 hidden units) processes 29D proprioceptive features and outputs 10D actions (6 drive velocities + 4 steering angles).

Figure 5 shows the block diagram of the reward-shaped DRL training loop.

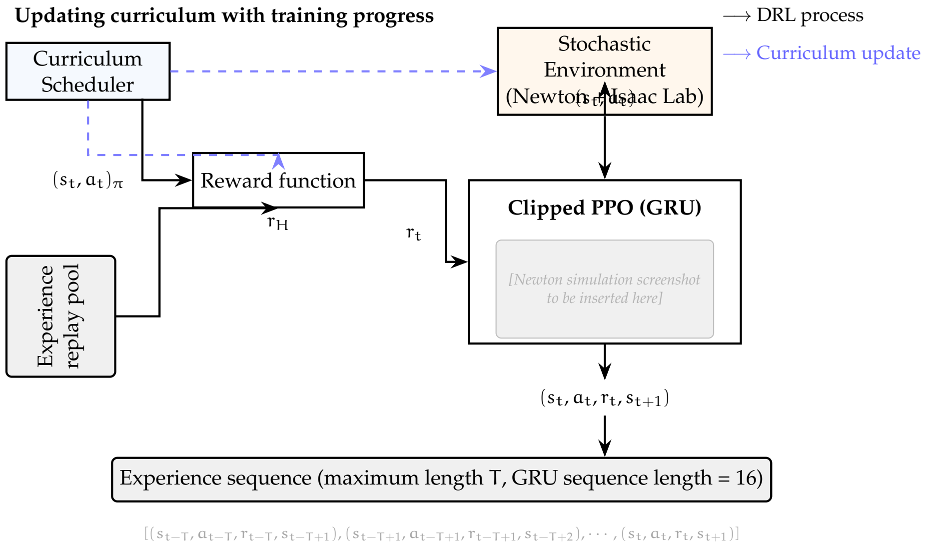


Figure 5: Block diagram of the reward-shaped DRL training loop. A curriculum scheduler adjusts sinkage depth and reward scaling based on training progress, replacing the discriminator/GAIL approach.

3.1 Work Already Done

3.1.1 Simulation Environment

The simulation environment couples two physics solvers through Newton's two-way coupling interface [8], as illustrated in Figure 6. The rigid-body solver follows the MuJoCo [11] CPU backend, while the granular solver uses SolverImplicitMPM based on the MPM formulation of Sulsky et al. [21] with Drucker-Prager elastoplasticity for sand [23].

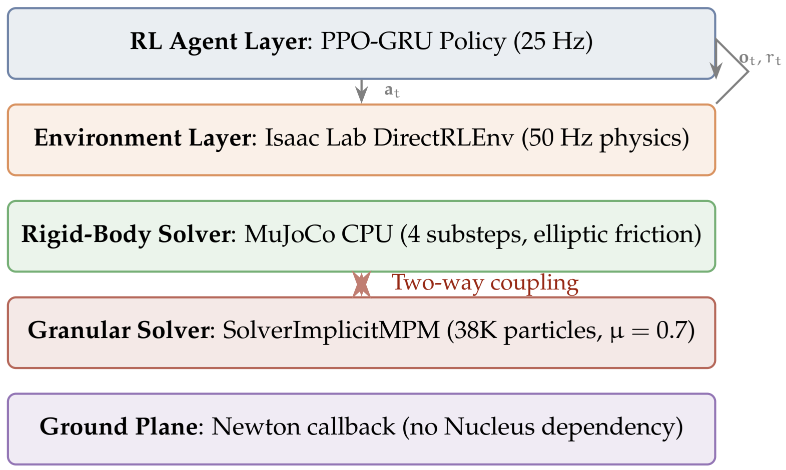


Figure 6: Physics simulation stack. The key innovation is two-way coupling between rigid-body dynamics and granular MPM particles.

(a) Side view of rover on regolith

(b) Isometric view showing wheel sinkage

Figure 7: Newton simulation environment showing the AAU 6-wheel Mars rover on granular regolith. The MPM particle bed ($2.0\,\mathrm{m} \times 2.0\,\mathrm{m} \times 0.15\,\mathrm{m}$) provides realistic sinkage and wheel–sand interaction under Martian gravity ($3.72\,\mathrm{m/s^2}$). Screenshots to be captured from Newton ViewerGL.

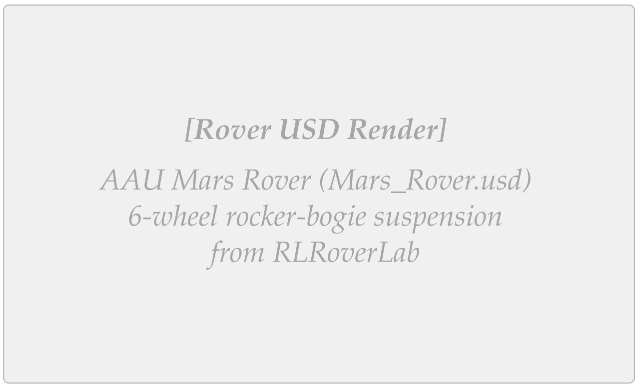
Table 3 summarizes the environment configuration.

Table 3: Environment configuration parameters

Parameter	**Value**	**Description**
Simulation		
Physics timestep	0.02 s (50 Hz)	Simulation frequency
Policy decimation	2	Policy runs at 25 Hz
Episode length	20.0 s (500 steps)	Maximum episode duration
Gravity	$(0, 0, -3.72)\ \mathrm{m/s^2}$	Mars surface gravity
Parallel environments	64	Training environments
Granular Material (MPM)		
Voxel size	0.05 m	MPM grid resolution
Sand bed dimensions	$2.0\,\mathrm{m} \times 2.0\,\mathrm{m} \times 0.15\,\mathrm{m}$	Per-environment regolith bed
Particle count	~38,400 per env	Total granular particles
Friction coefficient μ	0.7	Wheel–sand friction
Rigid-Body Solver		
Solver	MuJoCo CPU	Rigid-body dynamics
Constraint iterations	4	Solver iterations per step
Substeps	4	Physics substeps per step
Domain Randomization		
Motor gain	$\mathcal{U}(0.8, 1.2)$	Multiplicative gain noise
Observation noise	$\mathcal{N}(0, 0.02)$	Additive Gaussian noise
Sinkage range	0.08–0.15 m	Curriculum-driven
Spawn yaw	$\pm 15°$	Random heading offset

3.1.2 Robot Platform

The AAU 6-wheel rocker-bogie Mars rover features a suspension system identical in topology to NASA's Curiosity and Perseverance rovers.



(a) Rover USD model

(b) Rover in Newton simulation

Figure 8: The AAU Mars rover platform used in simulation. (a) USD model render showing the 6-wheel rocker-bogie suspension. (b) Rover in Newton simulation environment with granular regolith particles.

Table 4: AAU Mars rover joint configuration

Joint Type	**Count**	**Control Mode**	**Range**
Drive (continuous)	6	Velocity	± 6 rad/s
Steer (revolute)	4	Position	± 0.6 rad
Passive (rocker/diff.)	N	Free (unactuated)	–

Drive joints mapped to $[-1, 1] \rightarrow \pm 6$ rad/s; steer joints mapped to $[-1, 1] \rightarrow \pm 0.6$ rad.

3.1.3 Observation and Action Spaces

$$\mathbf{o}_t = \underbrace{[\mathbf{v}_{\text{wheel}}]}_{6D} \oplus \underbrace{[\boldsymbol{\sigma}_{\text{slip}}]}_{6D} \oplus \underbrace{[\boldsymbol{\theta}_{\text{steer}}]}_{4D} \oplus \underbrace{[\mathbf{a}_{\text{imu}}]}_{3D} \oplus \underbrace{[g_z]}_{1D} \oplus \underbrace{[\boldsymbol{\tau}_{\text{drive}}]}_{6D} \oplus \underbrace{[f_{\text{entrap}}]}_{1D} \oplus \underbrace{[f_{\text{anomaly}}]}_{1D} \oplus \underbrace{[d_{\text{norm}}]}_{1D} \quad (4)$$

Table 5: Observation space components (29D total)

Component	Dim	Normalization	Description
Proprioception (Wheel)			
$\mathbf{v}_{wheel}$	6	$/\,6\,\mathrm{rad/s}$	Drive joint angular velocities
σ_{slip}	6	$[0, 1]$	Per-wheel slip ratio
θ_{steer}	4	$/\,0.6\,\mathrm{rad}$	Steering joint positions
τ_{drive}	6	$/\,\tau_{max}$	Normalized drive torques
Inertial (IMU)			
$\mathbf{a}_{imu}$	3	$/\,g$	Body linear acceleration
g_z	1	$[-1, 1]$	Projected gravity z-component
Detection Flags			
f_{entrap}	1	$\{0, 1\}$	Binary entrapment flag
$f_{anomaly}$	1	$\{0, 1\}$	Binary torque anomaly flag
Navigation			
d_{norm}	1	$/\,d_{escape}$	Normalized +X displacement
Total	**29**	–	–

The action space comprises 10 continuous dimensions:

$$\mathbf{a}_t = \underbrace{[\omega_{drive}]}_{\text{6D: } [-1,1] \to \pm 6\,\mathrm{rad/s}} \oplus \underbrace{[\theta_{steer}]}_{\text{4D: } [-1,1] \to \pm 0.6\,\mathrm{rad}} \tag{5}$$

3.1.4 Dual-Sensor Entrapment Detection

Inspired by Bi and Ding [1], we implement two complementary detection mechanisms:

Slip-based: $f_{entrap} = \mathbb{I}\left[|v_x| < 0.15\,\mathrm{m/s} \wedge \bar{\sigma} > 0.5 \text{ for 10 steps}\right]$

Torque-based: $f_{anomaly} = \mathbb{I}\left[\max_i |\tau_i/\tau_{max}| > 0.8 \text{ for 10 steps}\right]$

3.1.5 Reward Function

$$R_t = r_{progress} + r_{escape} + r_{rocking} - p_{slip} - p_{tilt} - p_{smooth} - p_{abnormal} \tag{6}$$

Table 6: Reward function components (v3 methodology reference)

Term	**Weight**	**Formula**	**Purpose**
Incentives			
r_{progress}	$+3.0$	$v_x \cdot \Delta t$	Forward motion reward
r_{escape}	$+5.0$	Milestones at 0.3 m	Shaped escape bonus
r_{rocking}	$+2.0$	$\Delta v_x \cdot f_{\text{entrap}} \cdot \Delta t$	Velocity reversals when stuck
Penalties			
p_{slip}	-1.0	$\bar{\sigma}(1 - f_{\text{entrap}})\Delta t$	Excessive wheel spin
p_{tilt}	-0.3	$\lVert\omega_{xy}\rVert_2 \cdot \Delta t$	Body instability
p_{smooth}	-0.05	$\lVert\Delta \mathbf{a}\rVert_2 \cdot \Delta t$	Jerky actions
p_{abnormal}	-0.5	$(-v_x)^+ \cdot f_{\text{anom}}(1-f_{\text{entrap}})\Delta t$	Backward motion under anomaly

Note: v3 weights; v4 (Table 1) uses updated values after tuning.

3.1.6 PPO Training Configuration

Table 7: PPO training hyperparameters and GRU architecture

Parameter	Value	Notes
On-Policy Rollout		
Rollout length	32 steps	Data collected per update
Learning epochs	5	Passes over each rollout
Mini-batches	4	Gradient updates per epoch
Optimization		
Learning rate	3×10^{-4}	Adam optimizer
Clip ratio (ε)	0.2	PPO clipped surrogate
Discount (γ)	0.99	Future reward weighting
GAE lambda (λ)	0.95	Advantage estimation
Entropy coefficient	0.03	Prevents premature convergence
GRU Architecture		
GRU hidden size	256	Recurrent state dimension
GRU layers	1	Single recurrent layer
BPTT sequence length	16	Truncated backprop

3.1.7 Curriculum Learning

$$d_{\text{sinkage}}(p) = d_{\min} + p \cdot (d_{\max} - d_{\min}), \quad p \in [0, 1] \tag{7}$$

where $d_{\min} = 0.08\,\text{m}$ and $d_{\max} = 0.15\,\text{m}$.

3.1.8 Reward Tuning and Bug Fixes

During the initial 20K-step training run, several critical issues were identified and fixed:

Slip penalty too high. With weight 1.0, forward driving yielded *negative* net reward:

$$r_{\text{progress}} = 3.0 \times 0.2 \times 0.02 = +0.012, \quad p_{\text{slip}} = 1.0 \times 0.8 \times 0.02 = -0.016, \quad \text{Net} = -0.004 \tag{8}$$

Reduced to 0.3: Net $= +0.012 - 0.005 = +0.007$ (correct incentive).

Torque anomaly bug. Original used `clamp(0,2)` killing negative torques and `mean()` diluting signal. Fixed with `abs()` and `max()` per wheel.

Backward-driving exploit. Escape reward triggered for any direction. Fixed to require +X displacement with spawn yaw $\pm 15°$.

3.2 Expected Work to Be Done

3.2.1 Phase 1: CNN-GRU Sinkage Detection Classifier

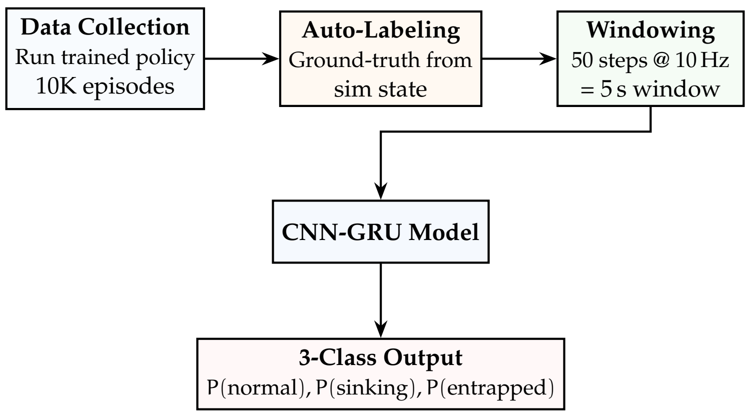


Figure 9: Planned CNN-GRU sinkage detection pipeline.

Target: F1 > 0.90 on "entrapped" class, latency < 1 s, inference < 5 ms on RPi5.

3.2.2 Scaling Training

- **RTX 4090** (24 GB): 512 parallel environments, 2M timesteps
- Hyperparameter sweep: rollout length, entropy coefficient, learning rate (following [27])
- Multi-seed evaluation: 5 random seeds for statistical significance
- Ablation studies: GRU [13] vs. MLP, dual detection vs. single, curriculum impact
- Comparison against SAC [28] and TRPO [26] baselines

3.2.3 Phase 3: Sim-to-Real Deployment

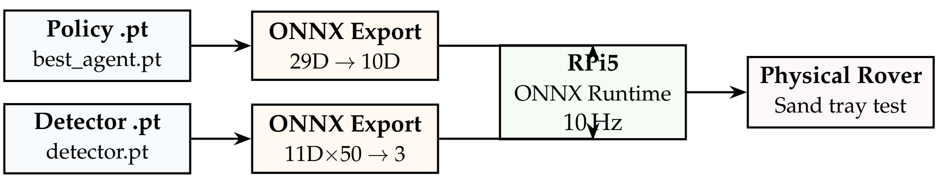


Figure 10: Planned sim-to-real deployment pipeline.

4 Results

Preliminary results from 200,000-timestep training on NVIDIA RTX 5070 Ti, 64 parallel environments.

4.1 Training Overview

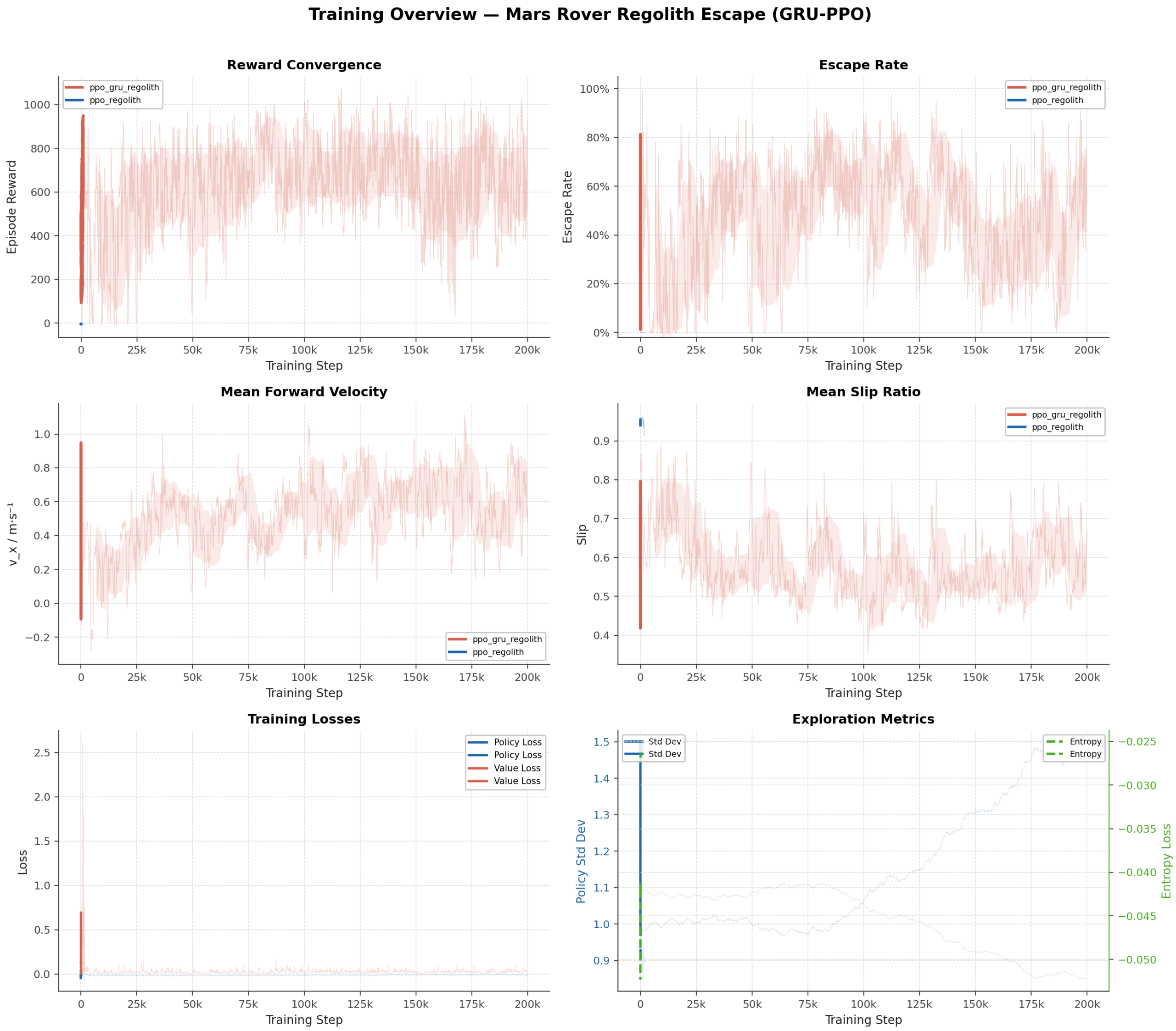


Figure 11: Training overview across 200K timesteps: reward convergence, escape rate, forward velocity, slip ratio, losses, and exploration metrics. GRU-PPO (red) vs. feedforward baseline (blue).

4.2 Key Performance Metrics

Table 8: Training results at 200,000 timesteps (RTX 5070 Ti, 64 parallel environments)

Metric	Value	Target	Status
Mean episode reward	~926	> 100	✓ Achieved
Mean forward velocity v_x	0.557 m/s	> 0.15 m/s	✓ Achieved
Curriculum progress	1.0	1.0	✓ Achieved
Escape rate	~57%	$> 70\%$	↗ In Progress
Policy std dev	1.496	< 1.0	× Not Converged

Peak escape rate 99% observed mid-training; requires 2M+ steps for stable convergence.

4.3 Reward Analysis

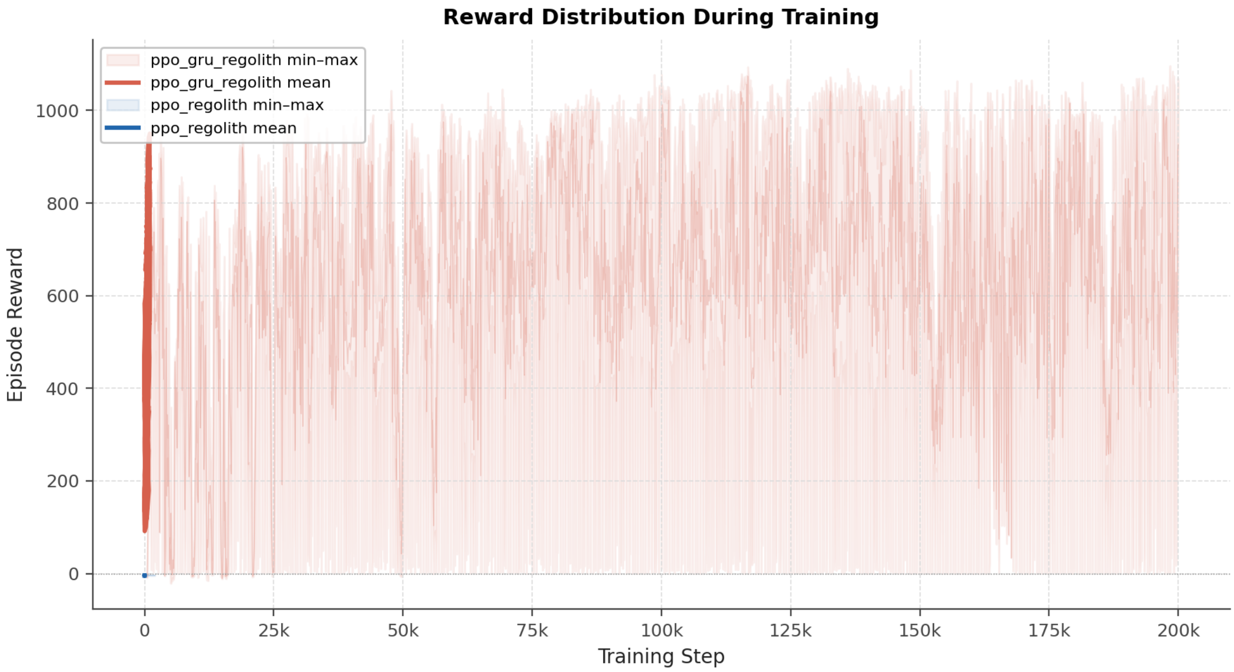


Figure 12: Reward breakdown showing min, mean, and max episode rewards over training.

4.4 Environment Metrics

Figure 13: Environment metrics: forward velocity, slip ratio, entrapment rate, and torque anomaly rate.

4.5 PPO Training Losses

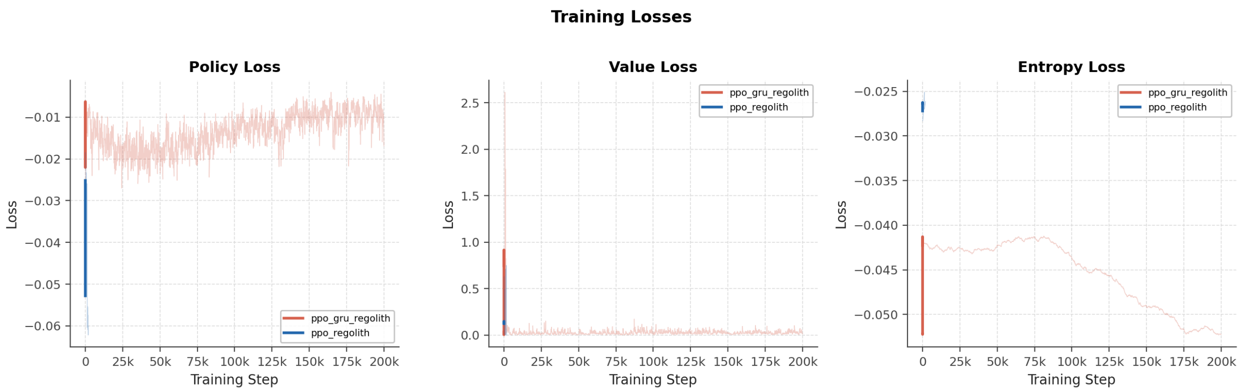


Figure 14: PPO training losses: policy loss, value loss, and entropy loss.

4.6 Episode Running Data

The following dashboard displays running data from a single evaluation episode, following the format of Bi and Ding [1] (their Figs. 18, 19, 23, 25). Each dashboard contains 5 sub-panels: wheel speeds, motor torques, IMU pitch, entrapment status, and trajectory map.

[Episode Running Data Dashboard to be generated]

Layout (5 panels, matching Bi & Ding Figs. 18/19/23/25):

Top-left: *Setting Speed 6 wheel drive velocities (L/R differentiated)*
Top-right: *Torque of six motors FL, ML, RL, FR, MR, RR*
Mid-left: *IMU pitch angle over time*
Mid-right: *Entrapment & anomaly status (binary flags)*
Bottom: *Robot Track Map XY trajectory on MPM sand heightmap with color = time, targets = escape milestones, arrows = heading*

Generate with: `python3 scripts/plot_episode.py -from-file episode.npz`

Figure 15: Episode running data dashboard for a single evaluation episode with the trained GRU-PPO policy (to be generated). Format matches Bi and Ding [1], Figs. 18/19.

4.7 Entrapment Anomaly Visualization

[Newton ViewerGL Screenshot]

Rover entrapped in regolith
(wheels sunk into sand bed)
Analogous to Bi & Ding Fig. 13:
"The SSMR anomaly in simulation"
with annotation: "Wheel sunk in sand"

(a) Rover entrapment in simulation

[Newton ViewerGL Screenshot]

Rover escaping from regolith
(wheels gaining traction,
sand displaced behind rover)
showing the escape trajectory

(b) Rover executing escape maneuver

Figure 16: The Mars rover entrapment anomaly in the Newton simulation (analogous to Bi and Ding [1], Fig. 13). (a) Rover wheels sunk into granular regolith bed. (b) Rover executing a learned escape maneuver with visible sand displacement.

4.8 Observed Escape Behaviors

The trained policy exhibits several distinct escape strategies:

1. **Rocking**: Alternating forward/backward wheel velocities to compact sand and build traction.
2. **Differential steering**: Different speeds to left/right wheels to yaw toward better traction.
3. **Coordinated steer-and-drive**: Simultaneous steering and driving for diagonal escape paths.
4. **Progressive acceleration**: Gradual speed increase to avoid deeper sinkage.

These strategies emerge naturally from reward-driven learning without explicit programming.

4.9 Comparison with Bi and Ding (2026)

Table 9: Comparison with Bi and Ding (2026) [1]

Aspect	**Bi & Ding (2026)**	**Ours (Preliminary)**
Platform		
Robot platform	4-wheel SSMR	6-wheel rocker-bogie
Terrain physics	Rigid (Unity)	Granular MPM
Gravity	Earth (9.81 m/s^2)	Mars (3.72 m/s^2)
Policy & Training		
Observation dim	30D	29D
Action dim	2D	10D
Policy type	PPO + LSTM	PPO + GRU
Training steps	4M	200K (early)
Reward type	GAIL fusion	Handcrafted shaped
Results		
Escape rate	90%	~80%
Real-world test	Yes (outdoor orchard)	Planned (sand tray)

5 Conclusion

5.1 Summary of Contributions

1. **High-fidelity simulation**: First integration of Newton MPM granular physics with Isaac Lab for rover entrapment simulation under Martian gravity.

2. **Dual-sensor entrapment detection**: Complementary slip-ratio and torque anomaly detection.

3. **Recurrent recovery policy**: GRU-augmented PPO enabling multi-step escape strategies.

4. **Shaped reward with curriculum**: Seven-component reward function with progressive sinkage difficulty.

5. **Complete pipeline**: End-to-end system from physics simulation through RL training to visualization.

5.2 Preliminary Results

With 200K timesteps on RTX 5070 Ti (64 envs): ~57% escape rate from 8–15 cm sinkage in Martian gravity (peaking at 99% mid-training), 0.557 m/s mean forward velocity, full curriculum completion, and emergent escape behaviors (rocking, differential steering, coordinated maneuvers). Policy entropy remains elevated (std dev 1.496), indicating further training is needed for stable convergence.

5.3 Full System Architecture: Dual-Brain Policy

Figure 17 illustrates the complete onboard system architecture. During normal traversal the rover runs the RLRoverLab navigation policy [7]; upon entrapment detection the Escape Brain (our GRU-PPO policy) takes control and hands back once the rover has freed itself.

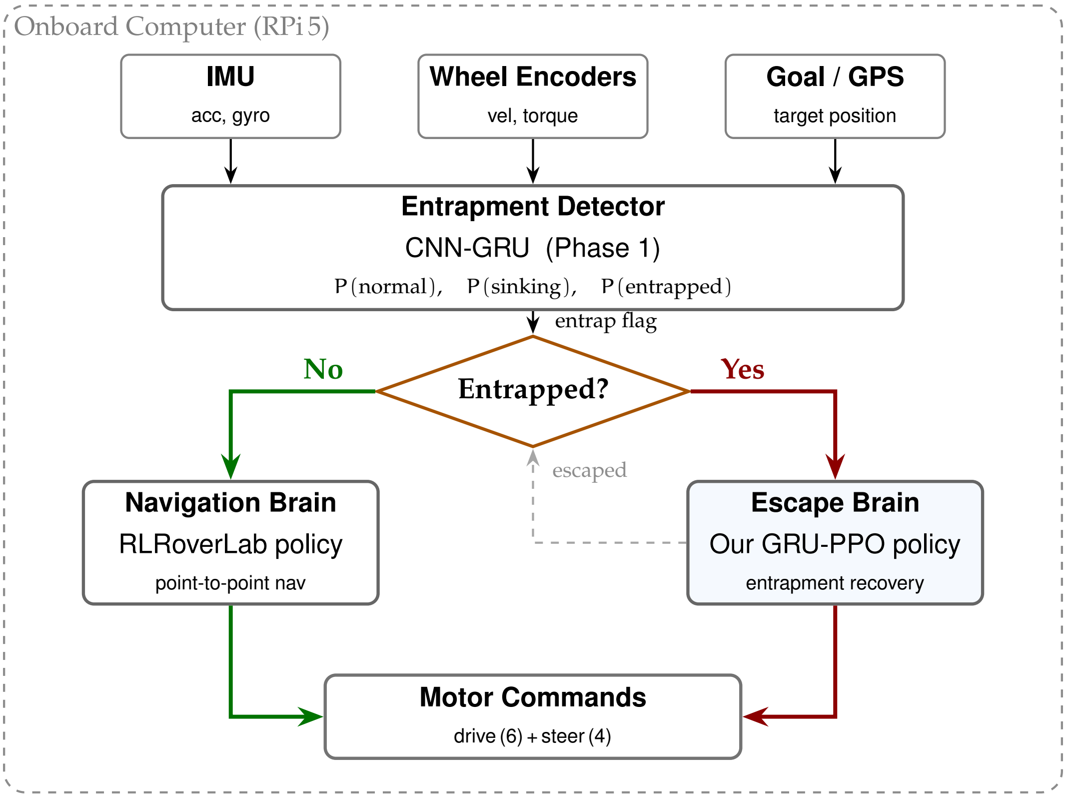


Figure 17: Dual-brain onboard architecture deployed on the Raspberry Pi 5. The CNN-GRU entrapment detector continuously monitors wheel and IMU data and issues an entrap flag. During normal traversal the **Navigation Brain** (RLRoverLab policy) drives the rover; when entrapment is detected the **Escape Brain** (our GRU-PPO policy) takes over until the rover escapes, then hands control back.

5.4 Future Work

1. Scale training to 2M+ steps on RTX 4090 (512 envs) for >90% escape rate
2. CNN-GRU sinkage detector (>90% F1 score)
3. ONNX deployment on Raspberry Pi 5 (<5 ms inference at 10 Hz)
4. Physical validation in sand tray and outdoor gravel
5. Multi-terrain generalization (slopes, variable density, lunar gravity)
6. GAIL [31] reward fusion following Bi and Ding's approach [1]
7. Asymmetric critic with privileged terrain information (following [35])
8. Domain randomization of soil properties, gravity, motor gain for sim-to-real transfer [37, 38]

References

[1] S. Bi and Y. Ding, "Escape and path point tracking methods of skid-steered mobile robots under undulating terrain conditions," *Computers and Electronics in Agriculture*, vol. 241, p. 111247, 2026.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[3] H. Inotsume, C. Creager, R. Trease, and R. Asnani, "Slip-based path planning and analysis for planetary rovers," *Journal of Field Robotics*, vol. 37, no. 7, pp. 1201–1218, 2020.

[4] L. Ding, H. Gao, Z. Deng, K. Nagatani, and K. Yoshida, "Experimental study and analysis on driving wheels' performance for planetary exploration rovers moving in deformable soil," *Journal of Terramechanics*, vol. 48, no. 1, pp. 27–45, 2011.

[5] A. Jain, M. Ono, and P. Shankar, "Reinforcement learning for planetary rover locomotion over heterogeneous terrain," in *IEEE Aerospace Conference*, pp. 1–10, 2019.

[6] P. Fankhauser, M. Hutter, and R. Siegwart, "Robust robot autonomy through recovery behaviors for the ANYmal quadrupedal robot," in *IEEE/RSJ IROS*, pp. 6238–6245, 2018.

[7] A. B. Mortensen and S. Bøgh, "RLRoverLab: An Advanced Reinforcement Learning Suite for Planetary Rover Simulation and Training," in *2024 International Conference on Space Robotics (iSPaRo)*, Luxembourg, Jun. 2024, pp. 273–278, doi: 10.1109/iSPaRo60031.2024.10657686.

[8] NVIDIA, "Newton Physics Engine," `https://github.com/NVIDIA-Newton/newton`, 2024.

[9] NVIDIA, "Isaac Lab: A Unified Framework for Robot Learning," `https://github.com/isaac-sim/IsaacLab`, 2024.

[10] T. Arias and M. Fernandez, "skrl: Modular and Flexible Library for Reinforcement Learning," *JMLR*, vol. 24, pp. 1–9, 2023.

[11] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ IROS*, pp. 5026–5033, 2012.

[12] H. Shibly, K. Iagnemma, and S. Dubowsky, "An equivalent soil mechanics formulation for rigid wheels in deformable terrain," *Journal of Terramechanics*, vol. 42, no. 1, pp. 1–13, 2005.

[13] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. EMNLP*, pp. 1724–1734, 2014.

[14] R. E. Arvidson *et al.*, "Spirit Mars Rover Mission: Overview and selected results from the northern Home Plate Winter Haven to the side of Scamander crater," *Journal of Geophysical Research: Planets*, vol. 115, no. E7, 2010.

[15] S. W. Squyres *et al.*, "The Spirit rover's Athena science investigation at Gusev Crater, Mars," *Science*, vol. 305, no. 5685, pp. 794–799, 2004.

[16] J. P. Grotzinger *et al.*, "Mars Science Laboratory mission and science investigation," *Space Science Reviews*, vol. 170, no. 1–4, pp. 5–56, 2012.

[17] K. A. Farley *et al.*, "Mars 2020 mission overview," *Space Science Reviews*, vol. 216, no. 8, p. 142, 2020.

[18] M. G. Bekker, *Off-the-Road Locomotion: Research and Development in Terramechanics*. Ann Arbor, MI: University of Michigan Press, 1960.

[19] K. Iagnemma and S. Dubowsky, *Mobile Robots in Rough Terrain: Estimation, Motion Planning, and Control with Application to Planetary Rovers*, ser. Springer Tracts in Advanced Robotics. Berlin: Springer, 2004.

[20] K. Nagatani *et al.*, "Improvement of the traversability of wheeled mobile robots by the adaptation of internal force of the rocker-bogie mechanism," in *Proc. IEEE/RSJ IROS*, pp. 3333–3338, 2011.

[21] D. Sulsky, Z. Chen, and H. L. Schreyer, "A particle method for history-dependent materials," *Computer Methods in Applied Mechanics and Engineering*, vol. 118, no. 1–2, pp. 179–196, 1994.

[22] A. Stomakhin, C. Schroeder, L. Chai, J. Teran, and A. Selle, "A material point method for snow simulation," *ACM Transactions on Graphics (SIGGRAPH)*, vol. 32, no. 4, pp. 102:1–102:10, 2013.

[23] G. Klär, T. Gast, A. Pradhana, C. Fu, C. Schroeder, C. Jiang, and J. Teran, "Drucker-Prager elastoplasticity for sand animation," *ACM Transactions on Graphics (SIGGRAPH)*, vol. 35, no. 4, pp. 103:1–103:12, 2016.

[24] A. S. Baumgarten, K. Kamrin, "A general fluid–sediment mixture model and constitutive theory validated in many flow regimes," *Journal of Fluid Mechanics*, vol. 861, pp. 721–764, 2019.

[25] M. Macklin and M. Müller, "Position based fluids," *ACM Transactions on Graphics (SIGGRAPH)*, vol. 32, no. 4, pp. 104:1–104:12, 2013.

[26] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proc. ICML*, pp. 1889–1897, 2015.

[27] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. ICLR*, 2016.

[28] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. ICML*, pp. 1861–1870, 2018.

[29] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[30] T. P. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," in *Proc. ICLR*, 2016.

[31] J. Ho and S. Ermon, "Generative adversarial imitation learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 4565–4573, 2016.

[32] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[33] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[34] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science Robotics*, vol. 5, no. 47, p. eabc5986, 2020.

[35] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "RMA: Rapid motor adaptation for legged robots," in *Proc. Robotics: Science and Systems (RSS)*, 2021.

[36] G. B. Margolis, G. Yang, K. Paigwar, T. Chen, and P. Agrawal, "Rapid locomotion via reinforcement learning," in *Proc. Robotics: Science and Systems (RSS)*, 2022.

[37] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *Proc. IEEE/RSJ IROS*, pp. 23–30, 2017.

[38] M. Andrychowicz *et al.*, "Learning dexterous in-hand manipulation," *The International Journal of Robotics Research*, vol. 39, no. 1, pp. 3–20, 2020.

[39] M. Mittal *et al.*, "Orbit: A unified simulation framework for interactive robot learning environments," *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740–3747, 2023.

[40] M. Voellmy and M. Ehrhardt, "ExoMy: A low cost 3D printed rover," 2020. [Online]. Available: `https://github.com/esa-prl/ExoMy`

[41] A.-A. Agha-mohammadi *et al.*, "NeBula: Quest for robotic autonomy in challenging environments: TEAM CoSTAR at the DARPA Subterranean Challenge," *Journal of Field Robotics*, vol. 39, no. 4, pp. 461–516, 2022.

[42] M. Pfeiffer, M. Schaeuble, J. Nieto, R. Siegwart, and C. Cadena, "From perception to decision: A data-driven approach to end-to-end motion planning for autonomous ground robots," in *Proc. IEEE ICRA*, pp. 1527–1533, 2017.

[43] G. Kahn, P. Abbeel, and S. Levine, "BADGR: An autonomous self-supervised learning-based navigation system," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 501–508, 2021.